

hermetic_netns_poc.sh — companion guide (§11.4.18)

Revision: 2 **Last modified:** 2026-07-02T00:00:00Z **Status:** H0
feasibility proof for the hermetic WireGuard test harness ([design](#)).
Authority: inherits constitution/Constitution.md per §11.4.35.

Overview

tests/vpn_lan/hermetic_netns_poc.sh proves — rootlessly, deterministically, and with real captured evidence — that the network-namespace substrate under the planned hermetic WireGuard harness works on this host. It stands up **two network namespaces joined by an L3 veth link**, serves a **real HTTP payload** (a fresh per-run nonce) from the peer namespace, fetches it from the other side, and asserts the **sha256 matches byte-for-byte**. Everything runs inside a single unshare -Urnm (user + net + mount namespace), so the caller needs **no root and no host privilege** — unshare -Ur maps the caller to root *inside* the throwaway user namespace, where ip/nsenter operations are permitted, and every interface/process created is confined to that namespace and torn down when unshare exits.

Honest scope (§11.4.6): this PoC uses a **veth** pair, not yet WireGuard. It proves the netns + rootless + L3-round-trip feasibility — the exact shape the userspace-WireGuard tunnel (wireguard-go) occupies in H0-full. It does **not** itself exercise WireGuard, and it does **not** prove the real Mullvad topology (that stays a §11.4.3 operator-gated confirmation).

Prerequisites

- bash, util-linux unshare + nsenter, iproute2 ip, python3, sha256sum, timeout (self-bounds the peer server so an outer-SIGKILL orphan terminates on its own — no indefinite linger, §12).
- The host must permit **unprivileged user namespaces** (kernel.unprivileged_userns_clone = 1 where the knob exists; user.max_user_namespaces / user.max_net_namespaces > 0).
- Enough process headroom that forking ~4 short-lived processes is safe.

When any prerequisite is missing the script **SKIPS honestly** (exit 0, a SKIP: line naming the reason) — never a fake PASS (§11.4.3).

Usage

```
tests/vpn_lan/hermetic_netns_poc.sh          # PASS / SKIP / FAIL
POC_MUT=1 tests/vpn_lan/hermetic_netns_poc.sh
# §1.1 golden-bad – PASS iff the teeth caught the tamper
```

Evidence is written to qa-results/vpn_lan/hermetic/<UTC-ts>_<pass|mut>_<pid>/roundtrip.evidence.

Edge cases

- **Unprivileged users disabled** → SKIP: ... unprivileged user namespaces disabled.
- **Low process headroom** (`ulimit -u` minus in-use < 64) → SKIP: ... §12 host-safety (refuses to add fork pressure to a starved host).
- **Missing tool** → SKIP: ... tool absent: <name>.
- **Setup race** (holder netns not ready) is prevented by a marker the holder touches *from inside its swapped netns*; if the marker never appears the run FAILs honestly rather than silently mis-configuring.
- **POC_MUT=1** tampers the served payload *after* the source hash is fixed; the only acceptable outcome is a FAIL at the sha256/body mismatch check — any other failure mode is reported as "teeth not proven" (§11.4.107(10)).

Internal behaviour

Outer stage: preflight (tools + users kill-switch + unshare -Ur -n true smoke

- process-headroom guard), then re-exec "\$0" --inner under unshare -Urm, then classify the verdict from the captured evidence. Inner stage: veth0/veth1 pair, a held-child peer netns (unshare -n ... &) referenced by PID, veth1 moved into it, 10.9.0.1/10.9.0.2 addressing, a python3 -m http.server bound to the peer address, a readiness poll, then the fetch + sha256 comparison.

Captured evidence (verified 2026-07-02)

- **Determinism (§11.4.50):** 3/3 clean PASS, each with a distinct sha256 from a distinct nonce (8b752cce..., 95f3bc06..., fddb1bd6...) — real fresh round-trips, not a frozen result (§11.4.107).
- **Teeth (§11.4.107(10)):** POC_MUT=1 → exit 1 at sha256/body mismatch — the assertion is load-bearing.
- **Feasibility probe:** kernel.unprivileged_userns_clone=1, max_user/net_namespaces=255793, unshare -Ur -n rc=0, /dev/net/tun present.

Related scripts

- [hermetic_wg_test_harness.md](#) — the design this seeds.
- tests/lib/sword_bridge.sh — the HELIX_BRIDGE_MODE=hermetic integration target (H2).
- tests/vpn_lan/sword_bridge_unit.sh — sibling anti-bluff VPN-LAN unit test.

Last verified

2026-07-02 (host: ALT Linux kernel 6.12, util-linux 2.39.2, python3 present).